

How Do You Audit a Computation You Cannot See?

Verifying which model an inference provider actually ran: what works, what it costs, and how easily the measurement lies to you

Jiawei Li

A summary of the `inference-verification` (`ivgym`) experiments,
with new measurements run for this report on a single NVIDIA H100-80GB

August 2026

Abstract

A treaty clause that caps training compute, a lab’s commitment to its regulator that a deployed model is frozen, and an inference bill reduce to the same unverifiable sentence: *model M ran under spec ϕ* — on hardware the auditor cannot enter. Nothing stops the party controlling that hardware from quietly serving a 4-bit copy, a smaller model, or a different sampler. This report summarizes an experimental study of whether an outside verifier, holding only what crosses the boundary — the tokens that came back, and when they arrived — can catch that, using a testbed (`ivgym`) that pits provider-side deviations against verifier-side detectors on real models on a GPU.

Four findings organize everything else. First, **the honest verifier works, but only barely, and at a price**: recomputing the reference model and comparing per-token sampling margins catches every deviation we tried on some model, but no single detector is reliable across model families, and the per-token signal from realistic quantization is so weak ($d' \approx 0.07$) that thousands of tokens must be pooled before a decision is trustworthy. Second, **most of the cheap alternatives fail in a specific and predictable place**: a verifier-side proxy model catches wholesale model substitution at AUC 0.998 without ever running the reference, but is blind to quantization, because quantization moves the served distribution less than the reference model’s own run-to-run nondeterminism. Third, **the arrival times of a stream already bought are a second verifier, and the only free one**: a token cannot arrive before its bytes have moved, so the deviations that read fewer bytes leave a timing trace that costs the auditor zero FLOPs and needs nothing from the audited party. Measured on real hardware, every *absolute* latency test fails — the per-token floor sits on a 4.15 ms serving-stack constant that appears on no spec sheet — but a *differential* test, subtracting a short-context probe’s inter-token time from a long one’s, cancels the stack, cannot be pushed toward a false accusation by co-tenancy, and catches a provider truncating its billed context at 36–152 tokens of stream, scored through the same protocol as every other detector here. Fourth, and least expected: **the dominant error in this literature-adjacent measurement is statistical, not cryptographic**. Resampling evaluation batches from too small a token pool inflates detection AUC arbitrarily; we show the same measurement reading 0.98 and 0.53 depending only on pool size, and re-measure the testbed’s headline table inside a valid regime, where 16 of 24 cells fall by up to 0.46 AUC and two apparent detector capabilities turn out not to exist. The same correction dissolves the study’s reported “detector reversals”: re-measured, none of them has a negative per-token effect size, and the detectors that appeared to point the wrong way are simply at chance. On a 56,160-token pool — large enough to test the prediction for the first time — the per-token effect size forecasts the batch at which detection arrives to within 0.03 AUC.

We also report two negative results that we believe generalize. A learned, calibrated per-token “value” head and a formally-derived information-directed matched filter both fail to beat

a hand-written entropy heuristic, while a hardware-aware prefix scheduler — which is pure cost geometry and contains no statistics at all — gives a 16× wall-clock saving. In selective verification, the cost side transfers and the signal side does not.

1 The problem

Three instruments that look nothing alike reduce to one sentence. A treaty that caps training compute is a promise about which computations ran on a fleet. A commitment to a regulator that a deployed model is frozen is a promise about which computation answers each request. An inference bill is a promise that a specific model ran under a specific spec. In all three, the party making the promise controls the hardware, and the party checking it cannot enter the building: nothing in the stack between a GPU and the outside world emits a statement about what the silicon actually did, so an auditor holds only what crosses the boundary — the tokens that came back, and when they arrived. This report measures what those two observables can prove, and what a verdict costs in each.

We study the tractable instance of that problem, because it is the one where every claim can be tested on real hardware and the deviations have the same shape one level up. An inference provider claims a *specification* ϕ : a model M and its sampling parameters (temperature, top- k , top- p , and a seed). It returns (**prompt**, **output**) pairs and asserts they came from running ϕ on M . It may instead have run something cheaper. The deviations worth making are not exotic — each is pure margin for the party running the hardware:

- **Serve a smaller model.** Bill for a 4B, serve a 0.6B. This is the highest-value cheat — here, 6.7× fewer FLOPs.
- **Quantize.** Serve the right model at 4-bit or with an fp8 KV cache. Subtle, and cheap to do.
- **Change the sampler.** Wrong temperature, wrong seed, or a plain bug that truncates the candidate set.

The auditor cannot inspect the provider’s hardware, and the provider is exactly the party whose self-reports cannot be trusted — so any verifier that needs the provider’s cooperation (a reported trace, a logged KV cache) is one the provider can stub out. Zero-knowledge proofs of inference solve this in principle and are many orders of magnitude too slow in practice. What remains are the two passive observables above: most of this report reads the tokens (Sections 3–5), and Section 6 reads the clock.

The token-side approach follows DiFR [1]: the verifier *recomputes*. It runs M itself over **prompt** + **claimed_tokens**, and asks whether each claimed token is what ϕ on M would have produced. The complication that makes this a research problem rather than an equality check is **nondeterminism**: batching, kernel selection, and reduction order all change logits slightly, so honest outputs do not reproduce bit-exactly. Verification is therefore a hypothesis test, not a comparison.

What this report is. The `ivgym` testbed makes the two sides of that test pluggable — attacks (provider deviations) and defenses (per-token divergence scores) — and holds the evaluation protocol fixed, so “who catches whom” is a grid rather than an anecdote; the design follows the model-organism testbed pattern [4], in which known deviations are planted so that a detector’s misses are observable. This report summarizes what that grid says across roughly two dozen experiments on Qwen3, Llama-3.x, SmolLM2, Pythia and GPT-2 models, and adds new measurements, run for this report, that close the largest gaps in the existing record — including a measured study of the second observable, the clock (Section 6), scored through the same protocol as the token detectors. New results are marked [**new**].

2 Setup: the game, and the number we report

The players. The **provider** generates under a spec of its choosing while claiming ϕ . The **verifier** recomputes under ϕ and emits a per-token divergence score $s_t \geq 0$, higher meaning “less consistent with ϕ on M ”. The verifier never sees the attack. Provider and verifier share a per-position seed for the Gumbel noise used in sampling, so under the default detector an honest token scores exactly 0 whenever the verifier’s own argmax agrees with the claimed token — which is the common case, about 98% of honest tokens — and a positive post-Gumbel margin otherwise.

Four detectors. `token_difr` is the post-Gumbel margin above; `cross_entropy` is the negative log-likelihood of the claimed token; `activation_difr` is an L_2 distance on randomly projected activations; `token_toploc` is the rank of the claimed token in the verifier’s filtered distribution (no seed synchronization needed). All four require recomputing M . A separate family of *black-box* detectors scores (`prompt`, `tokens`) without running M at all, using a small proxy model or no model.

From a token score to a decision. A single token is far too noisy to classify. The protocol is fixed once for every experiment: split honest tokens into disjoint calibration and evaluation halves; winsorize at the 99.9th percentile of the calibration half so one outlier cannot carry a batch; average b tokens into a batch statistic S ; and flag when $S > \tau$, with τ set to the $(1 - \alpha)$ quantile of the calibration half’s batch statistics.

The metric. We report the **standardized partial AUC at FPR $\leq 0.5\%$** . A provider is not a one-shot bet — it is checked continuously, so it cannot tolerate more than a sliver of false accusations. Restricting the ROC to that region and McClish-standardizing it keeps the familiar scale (0.5 = chance, 1.0 = perfect) while measuring only the operating region a verifier can actually use. It is consistently *stricter* than full-range AUC. Every number in this report is this quantity unless stated otherwise, and unless noted every new number is a mean $\pm$ standard deviation over 5 protocol seeds.

3 The measurement trap, and how to stay out of it

We put this first because it changes how to read every other table, including several published in the testbed’s own documentation.

3.1 Batches resampled from a small pool inflate AUC without bound

The batch statistic S is a mean of b tokens drawn *without replacement from a fixed token pool*. As b approaches the pool size, every batch mean converges to the pool mean and the honest variance collapses. The AUC then no longer answers “would a fresh batch be flagged?” — it answers “do these two particular pools have different means?”, which is nearly deterministic. Worse, it is deterministic in *both* directions: whichever pool mean happens to be larger drives the AUC toward 1.0 or toward 0.0.

The testbed’s own diagnostic makes this vivid on one cell. Holding the model, the attack, and the per-token scores fixed and growing only the pool, `token_difr` versus `quant_4bit` on Qwen3-1.7B reads:

honest pool (tokens)	batch / null split	AUC @ FPR $\leq 0.5\%$
576	69.4%	0.977 ± 0.010
1,152	34.7%	0.577 ± 0.042
2,304	17.4%	0.679 ± 0.099
4,608	8.7%	0.523 ± 0.021
9,216	4.3%	0.524 ± 0.012
22,464	1.8%	0.530 ± 0.011

0.977 and 0.530 are the same measurement. Note that the approach is not smooth: at 17.4% the reading bounces back to 0.679 with a ± 0.099 spread across protocol seeds. Over-ratio measurements are not conservatively biased, they are erratic, which is why the failure is hard to spot from a single table. The rule of thumb that falls out is to keep the batch well under $\approx 10\%$ of the honest evaluation split, and to watch the seed-to-seed spread as the tell.

The pool-independent way to state a deviation’s strength is the per-token standardized effect size $d' = (\mathbb{E}[s \mid \text{deviating}] - \mathbb{E}[s \mid \text{honest}]) / \text{sd}(s \mid \text{honest})$; a batch of b independent tokens separates by $d' \sqrt{b}$. For `quant_4bit` on this model $d' = 0.0716$, which is tiny, and which predicts that AUC 0.90 needs $b \approx 2,766$ — and hence a pool of at least 55,000 tokens to stay inside the ratio ceiling. That prediction was left untested, because no pool that large had been generated.

3.2 [new] The headline grid, re-measured inside the ceiling

The testbed’s front-page table — 6 attacks $\times$ 4 detectors on Qwen3-0.6B — was measured at a **78%** ratio, and had never been re-run. We generated a 117 prompt $\times$ 192 token pool (22,464 tokens per configuration) and scored every cell twice from the *same* per-token scores: once over the full pool at batch 1000 (an 8.9% ratio, legitimate), and once over a sub-pool of the first 20 sequences $\times$ 128 tokens at the same batch (a 78% ratio — the published configuration, reproduced as a sub-pool so that the pool size is the only thing that differs).

attack	token_difr		cross_entropy		activation_difr		token_toploc	
	78%	8.9%	78%	8.9%	78%	8.9%	78%	8.9%
quant_4bit	0.908	0.718	0.565	0.505	1.000	1.000	0.732	0.600
kv_fp8	0.576	0.511	0.597	0.507	1.000	1.000	0.679	0.517
temp_1.1	0.596	0.516	0.897	0.823	0.551	0.504	0.808	0.661
seed_43	1.000	1.000	0.987	0.524	0.499	0.575	0.828	0.507
bug_k2	0.695	0.553	0.499	0.510	0.618	0.499	0.501	0.523
bug_k32	0.992	0.712	0.590	0.605	0.499	0.499	0.912	0.686

Table 1: [new] The testbed’s headline grid, scored twice from the *same* per-token scores: at the published 78% batch/pool ratio and at a legitimate 8.9% ratio. Qwen3-0.6B, 22,464-token pool, batch 1000, mean over 5 protocol seeds (standard deviations in the text). **Bold** marks the detector a portfolio would pick for that attack — the same one in both arms, in every row. Figure 1A plots the same 24 cells.

Sixteen of the 24 cells fall, by a median of 0.137 and by as much as 0.463. Three are saturated at 1.000 in both arms, and five *rise* — the artifact’s other tail. Four of the five are within 0.08 of chance in both arms, which is what a collapsed-variance measurement pushed the wrong way looks like; the fifth is the one cell where `cross_entropy` has a real signal to report ($d' = 0.133$ on `bug_k32`), and there the small pool happened to *understate* it, 0.590 against 0.605. The artifact does not have a sign. Averaged over the grid, the seed-to-seed standard deviation is three times larger in

the inflated arm (0.041 versus 0.013). The small-pool measurement is not merely optimistic; it is unstable, because what it reports is a property of the particular pool it drew.

The re-measurement is conservative about what it overturns. The winning detector is unchanged in all six rows, so the portfolio recommendation of Section 9 survives intact. What changes is how good the winner is: the floor across the six attacks drops from 0.695 to 0.553, and `token_difr` — which at 78% clears AUC 0.90 on three of six attacks, the basis for calling it the strong general detector — clears it on one.

The valid arm is not just the pessimistic reading; it is the correct one. Two cells settle this. Under `seed_43` the provider redraws from *exactly* the honest distribution, so a detector that only reads where the claimed token sits in that distribution has, mechanistically, nothing to see. At 78%, `cross_entropy` reads 0.987 and `token_toploc` reads 0.828 — near-perfect detection of a deviation neither can in principle observe. At 8.9% they read 0.524 and 0.507, which is the answer the mechanism requires. On these cells the inflated arm did not exaggerate a real signal, it manufactured one. That is a sharper argument than “smaller numbers are safer”: the ratio ceiling is a correctness condition, and the grid contains its own falsification test for it.

One cell moves the other way and we cannot fully account for it. `activation_difr` on `seed_43` goes from 0.499 to 0.575 ± 0.021 , with a per-token $d' = +0.057$. `activation_difr` compares projected activations recomputed on the same claimed prefix, so the seed should be invisible to it. We attribute the residual to the honest and re-seeded pools containing different *text*, so the two nondeterminism floors are not sampled over identical contexts — a pool-composition effect rather than sampler sensitivity — but the ablation that would settle it was not run. The only genuine per-token reversals anywhere in the grid are `activation_difr` on `bug_k2` ($d' = -0.008$) and on `bug_k32` ($d' = -0.032$), and both are small enough to read as chance in both arms.

3.3 [new] Does the effect size predict when detection arrives?

Section 3 left a prediction untested for want of a pool: at $d' \approx 0.07$, `token_difr` versus `quant_4bit` should reach AUC 0.90 at a batch of roughly 2,800, which needs at least 55,000 honest tokens to stay under the ratio ceiling. We generated one — 117 prompts $\times$ 480 tokens on Qwen3-1.7B, **56,160 evaluation tokens**, a 28,080-token honest null split, 5 protocol seeds — and swept the batch across it. The measured per-token effect size is $d' = 0.0776$, which puts the AUC-0.90 batch at 2,361, inside the ceiling for this pool for the first time.

batch b	ratio	measured AUC	predicted	residual
200	0.7%	0.531 ± 0.007	0.520	+0.011
400	1.4%	0.559 ± 0.010	0.548	+0.011
800	2.8%	0.627 ± 0.015	0.625	+0.002
1,600	5.7%	0.781 ± 0.023	0.790	−0.010
2,361 [†]	8.4%	0.873 ± 0.008	0.900	−0.027
5,600	19.9%	0.996 ± 0.002	0.998	−0.003 <i>over ceiling</i>
11,200	39.9%	1.000 ± 0.000	1.000	+0.000 <i>over ceiling</i>

Table 2: [new] Detection arriving on schedule. Qwen3-1.7B, `token_difr` versus `quant_4bit`, one 56,160-token pool, batch swept, 5 protocol seeds. “Predicted” is $\Phi(d'\sqrt{b}/\sqrt{2})$ mapped through the same partial-AUC definition, using only the pool-independent per-token effect size. [†] the batch the effect size nominates for AUC 0.90. The last two rows exceed the 10% ratio ceiling and are shown for completeness. Plotted in Figure 1B.

The prediction holds. Across the five points inside the ceiling the mean absolute residual is

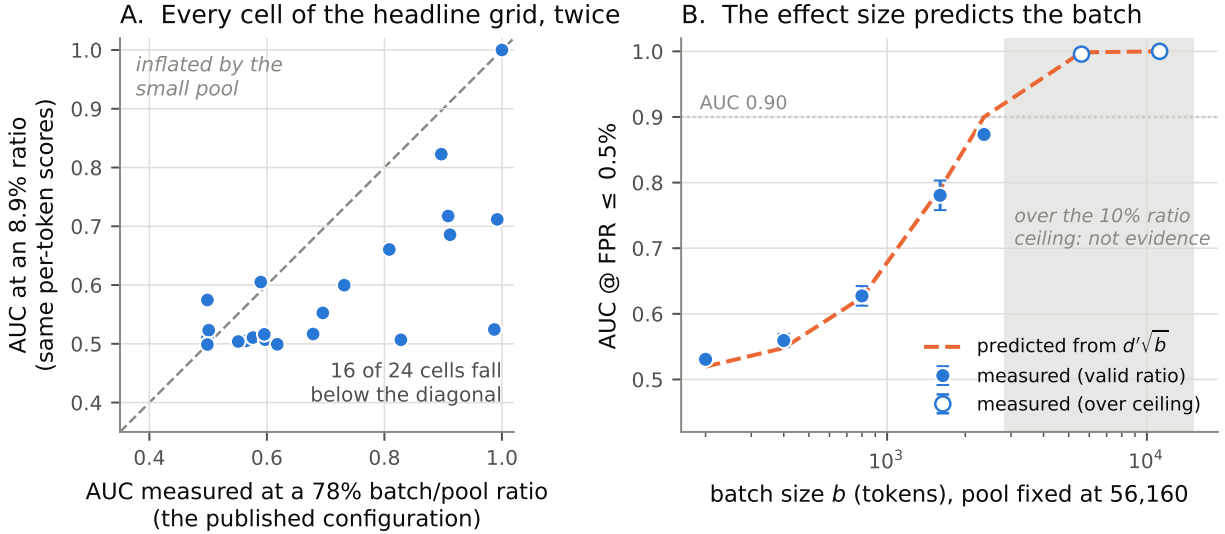


Figure 1: [new] The measurement trap, and the measurement that escapes it. **A:** every cell of Table 1, plotted at the published 78% ratio against the same per-token scores read at 8.9%. Points below the diagonal were inflated by the small pool. The handful above it are the same artifact acting in the other direction; all of them are within 0.08 of chance. **B:** Table 2 — with the pool held fixed at 56,160 tokens, the batch at which detection arrives is predicted by the per-token effect size alone, until the batch crosses the ratio ceiling and the measurement stops being evidence.

0.012 and the worst is 0.027, and at the nominated batch the measurement reads 0.873 ± 0.008 against a predicted 0.900. The residual is signed the way theory says it should be: the prediction treats the b tokens in a batch as independent, and tokens inside a sequence are not, so it should be mildly optimistic exactly where the curve is steep. Read as a design tool, the effect size gives a good estimate of the batch a verifier needs and a slight under-estimate of it. The two over-ceiling rows are consistent but uninformative — by then the true AUC is saturated, so the artifact has nothing left to inflate.

The deployment consequence is the blunt part. A 4-bit quantization of a 1.7B model is detectable — but the verifier needs on the order of 10^3 tokens pooled into each verdict and 10^4 – 10^5 honest tokens to calibrate the threshold against. That is not a few hundred tokens of one response; it is a continuous relationship with a provider, aggregated over many of them. Any protocol that promises a verdict on a single completion is not, on this evidence, measuring quantization.

3.4 [new] The “detector reversal” cells

The cross-family claim in Section 4 — no single recompute detector is uniformly robust — rests on its most dramatic cells: `token_difr` reading 0.040 on SmolLM2-1.7B under `kv_fp8` and 0.086 on Pythia-410m, cells where the divergence signal appears to *reverse*, the honest side scoring higher than the attacked side. Those cells come from a sweep run at a 52% ratio. A collapsed-variance measurement is pushed toward *both* extremes, so “some cells read 1.0 and some read 0.04” is precisely the artifact’s signature, and the evidence for the robustness claim has the same defect as the evidence it was raised against.

Reversal is a mechanistic claim, and it has a pool-independent test: the sign of the per-token effect size. A real reversal has $d' < 0$ no matter how the batches are drawn. We re-generated five cells — the two reversals, a third low cell, a mid-range cell, and a control the sweep read at 1.000 so

the check is two-sided — at 64 prompts $\times$ 128 tokens, and scored each three ways from a single generation pass.

model	attack	published	re-run at 52%	re-run at 9.8%	d'	reversed?
SmolLM2-1.7B	kv_fp8	0.040	0.601 ± 0.093	0.513 ± 0.008	+0.0188	no
pythia-410m	kv_fp8	0.086	0.499 ± 0.000	0.500 ± 0.001	+0.0011	no
pythia-410m	quant_4bit	0.146	0.501 ± 0.002	0.501 ± 0.001	+0.0101	no
Qwen3-0.6B	bug_k2	0.123	0.566 ± 0.073	0.515 ± 0.011	+0.0344	no
Qwen3-0.6B	seed_43	1.000	1.000 ± 0.000	1.000 ± 0.000	+0.6031	no

Table 3: [new] None of the reversals is real. Each cell re-generated at 8,192 tokens and scored three ways from one pass: at the sweep’s own 52% configuration (reproduced as a sub-pool so only the pool size differs), at a legitimate 9.8% ratio, and by the pool-independent per-token effect size d' . `token_difr`, 5 protocol seeds.

Every cell has $d' > 0$, including both cells the sweep reported as reversals: the attacked side scores higher than honest on all five, which is the direction the detector is designed to produce. Re-running the sweep’s own 52% configuration on a fresh pool reproduces neither published number — SmolLM2/kv_fp8 comes back at 0.601 ± 0.093 where the sweep printed 0.040 — and that ± 0.093 is itself the diagnosis. At 52% the seed-to-seed spread is an order of magnitude wider than at 9.8% (± 0.008), so 0.040 and 0.601 are two draws from the same badly-conditioned estimator rather than a disagreement about the world. The control behaves as a control must: `seed_43` reads 1.000 at both ratios, with $d' = +0.60$, because a large effect is a large effect at any pool size.

What survives is duller than the claim it replaces, and we think it is the more useful statement. `token_difr` does not invert on these models. It is simply *at chance* on them — 0.513, 0.500 and 0.515 at a valid ratio — because an fp8 KV cache at 8,192 tokens is below its detection floor, on every model we tried. That is the recompute-dominant-boundary result of Section 5 arriving from a second direction, not a robustness result. The portfolio argument keeps its mechanistic basis (each detector has a class of deviation it structurally cannot see, and the classes differ) and loses its most quotable evidence.

4 What recomputation catches

Read at a valid ratio, the detectors have a structure that is *mechanistic* rather than statistical — it follows from what each one is allowed to look at, and it is the part of the picture that no amount of re-measurement changes.

- **activation_difr is blind to the sampler, by construction.** It compares projected activations, which are computed before a token is drawn. Change the temperature, the seed, or the sampler itself and the activations are unchanged, so it sits at chance. Change the weights or the KV cache and it is the strongest detector available. This bifurcation is exact and reproducible, and it means `activation_difr` alone cannot verify a sampling specification.
- **cross_entropy needs the bulk of the distribution to move.** The likelihood of the claimed token is a coarse summary. A deviation that flips the top-2 tokens on 1% of positions (`bug_k2`) barely touches it, so it reads chance — while a temperature change, which moves every probability, is easy for it.
- **token_difr is the only general one**, because it sees both the forward pass and the draw: with a shared Gumbel seed, an honest token gives exactly zero and any deviation gives a positive margin. General is not the same as strong — the margin under kv_fp8 is reliably

positive and reliably tiny ($d' = 0.015$), which is why Sections 3.2 and 3.4 find it at chance there.

- **token_toploc is the practical surprise.** The rank of the claimed token in the verifier’s filtered distribution needs *no seed synchronization* — a significantly weaker deployment requirement — and tracks `token_difr` closely on most attacks. It shares `cross_entropy`’s blindness to `bug_k2`.

No single detector is robust across model families. A sweep over 9 models in 4 families (Qwen3, Llama-3.2, SmolLM2, Pythia; 0.36B–4B; three different tokenizers) $\times$ 7 attacks gives 63 cells per detector. `token_difr` averages 0.803, with cells at every level down to 0.040; the other three have floors at 0.000 (`cross_entropy`), 0.000 (`activation_difr`) and 0.013 (`token_toploc`). Taking the strongest detector per attack instead of betting on one score lifts the minimum detectability across all 63 cells to 0.78. That is the argument for treating detectors as a portfolio rather than picking a winner — and it is an argument the single-model tables cannot make, which is why a cross-family matrix is worth its GPU-hours. Averaged over models, the hardest attacks are `bug_k2` (0.580) and `kv_fp8` (0.639); a wrong seed is trivially caught (1.000 on every model), because it changes the realized draw everywhere while leaving the distribution alone.

That sweep ran 16 prompts $\times$ 96 tokens at batch 400, a **52%** ratio, so every absolute number in it — the 0.803 average, the 0.040 floor and the 0.78 portfolio minimum alike — sits well above the ceiling of Section 3 and should be read as an ordering rather than a level. Section 3.4 re-measures the cells that carried the strongest version of the claim, and they do not survive: what looked like a detector pointing the wrong way is a detector at chance, measured badly. The portfolio recommendation stands on the mechanism above — each detector is structurally blind to a different class of deviation — and not on the existence of reversals, which on present evidence there are none of.

5 Cheaper tiers, and exactly where each one stops

Recomputing M for every claimed token costs about as much as generating them. Three cheaper tiers were studied. They fail in the same place, for the same reason, and that common boundary is the most useful thing in this section.

5.1 Tier 0: no model at all, or a small proxy

Black-box detectors score (`prompt`, `tokens`) without running M : surface statistics, a small proxy model’s surprisal, or a zero-shot LLM judge reading the continuation. Their real value is as a *control*. If outputs alone can tell an attack from honest, the expensive recomputation is not buying information — it is duplicating a cheap signal. The clean case is a wrong seed: `token_difr` scores 1.000 while a zero-shot Claude judge sits at its own honest-null floor (0.511 against a floor of 0.989, i.e. no signal at all). Re-seeding redraws from the *same* distribution, so the continuation reads as a perfectly ordinary sample to any reader of the text, while a re-run flips which Gumbel candidate won. That gap is a quantitative proof that the deviation is invisible in the outputs and recomputation is doing real work. The converse is worth stating too: a *high* black-box AUC is not a verifier win, it is a statement that the attack was crude.

5.2 The one cheap signal with a trust story: the acceptance rate

The interesting Tier-0 detector borrows the mechanism of speculative decoding [3]. The verifier holds its own small proxy q (sharing M ’s tokenizer) and computes the rate at which q ’s drafts would have been accepted against the served distribution p . That rate is exactly an overlap:

$$\text{accept} = \mathbb{E}_{x \sim q} \left[\min \left(1, \frac{p(x)}{q(x)} \right) \right] = \sum_x \min(p(x), q(x)) = 1 - \text{TV}(p, q).$$

The verifier *owns* q , so q is a trusted anchor even though p is not. If the provider corrupts the target ($p^* \rightarrow \hat{p}$), the rate moves from $1 - \text{TV}(p^*, q)$ to $1 - \text{TV}(\hat{p}, q)$, and the auditor sees the shift against a one-time offline honest reference — without ever running M . Everything this tier can and cannot do follows from one budget: the expected proxy surprisal on honest text is $H(M, q) = H(M) + \text{KL}(M \| q)$, so $\text{KL}(M \| q)$ is the *entire* signal available to any proxy detector.

- **It wins outright on substitution.** A provider paid for Qwen3-4B that secretly serves Qwen3-0.6B is caught at **AUC 0.998** (honest acceptance 0.776 versus 0.700), using a trusted 1.7B draft and never running M — at $2.3\times$ fewer FLOPs than recomputation. Swapping the model moves $\text{TV}(p, q)$ wholesale.
- **It survives the standard evasion.** A provider that quantizes and then retunes temperature to restore the honest *entropy* blinds an entropy or cross-entropy fingerprint (AUC $1.00 \rightarrow 0.59$) but cannot re-enter the honest acceptance band, because matching entropy does not restore $\text{TV}(\hat{p}, q)$. The draft-anchored axis still separates it (0.998).
- **It is blind to realistic quantization.** At `quant_4bit` strength the measured acceptance-rate AUC sits near chance on real models, while `token_difr` separates every attack. Quantization moves $\text{TV}(p, q)$ less than the reference model’s own honest run-to-run nondeterminism.

This is the **recompute-dominant boundary**, and every cheap tier in the study hits the same wall: *deviations that change what the model can do are cheap to catch; deviations that change only how precisely it does it are not*. A cheap proxy shrinks how often the exact recomputation must fire. It does not replace it.

5.3 A richer proxy panel, and the dose-response curve

A separate line of experiments makes the capability-detection idea quantitative on a larger claimed model. A provider claims Qwen2.5-7B-Instruct; a verifier holding only sub-2B proxies scores the completion with eight aggregate features (surprisal mean/sd/max, mean log-rank, fractions of surprising ranks, mean entropy) from a single teacher-forced pass, and a logistic detector is cross-validated over 300 Alpaca prompts. Detectability tracks how much compute the attacker saved:

served instead of the 7B	compute gap	panel AUC @ 128 tok	verdict
Qwen2.5-0.5B	15.4×	0.997	caught
Qwen2.5-1.5B	4.9×	0.991	caught
Qwen2.5-3B	2.5×	0.580	near chance
Qwen2.5-7B in nf4	1.0×	0.462	chance

The cliff sits between a $5\times$ and a $2.5\times$ downgrade, at a verification cost of about 27% of *one* claimed-model forward pass. This detector also proved hard to evade by sampling tricks: an attacker tuning temperature to match honest cross-entropy collapses a mean-cross-entropy detector to 0.65 while the panel holds at 0.99; searching a temperature $\times$ top- p grid for the panel’s weakest point

never pushes it below 0.995; paraphrasing the cheap model’s output with itself moves it from 0.997 to 0.990. Two benign perturbations of the honest provider’s numerical stack (kernel and batch changes; then fp32 instead of bf16) shift the features measurably — an honest-versus-honest classifier reaches 0.76 — but along a direction the capability detector ignores, so false positives stay at nominal (3.3% and 5.0% against a 5% budget) and detection is unaffected.

Substituting a model along an ordered “distance” ladder gives the same shape: a quantized copy of M is at chance to a cheap proxy, and AUC climbs monotonically with distance, crossing the useful range (≈ 0.85) only once the substitute has a different post-training lineage or a different family — and this holds even when the proxy’s tokenizer differs from M ’s.

A note on comparability. These panel numbers are full-range cross-validated ROC-AUC on greedy decoding over a fixed prompt set — a different and more forgiving protocol than the FPR-constrained partial AUC used everywhere else in this report. They should be read as a separate line of evidence about capability detection, not as entries in the same table.

6 [new] The second observable: the clock

Everything so far reads the returned tokens. The auditor’s other observable is *when* they arrived, and it has a property no token detector has: it costs zero audit FLOPs, adds zero latency, and needs nothing from the audited party — the verdict is read off the arrival times of a stream already bought. For a governance instrument, that is the difference between a check that can run on every request and one that must be budgeted; it is also the only channel here whose cost does not grow with the model being verified.

The premise. A batch-1 decode step is memory-bound: the provider must move every weight byte it claims and read every context position it billed for before a token can exist, so $t_{\text{token}} \geq \text{bytes}/\text{bandwidth}$. The deviations worth making are exactly the ones that read fewer bytes — serving a 0.6B as a 1.7B reads $2.83\times$ fewer, NF4 weights $2.55\times$, attending half a 32k context $1.35\times$ — while a wrong seed, a wrong temperature or a top- k bug reads *identical* bytes ($1.00\times$). The provider’s saving and the auditor’s evidence are the same quantity, and the channel’s blindness class is the exact complement of the token channel’s: the sampler deviations the clock cannot see are the ones recomputation catches cheapest (a wrong seed at 226 tokens), and the byte deviations the token channel reads worst (an fp8 KV cache at 35,066 tokens) are the ones with a timing trace.

Measured, every obvious clock test fails. We timed 126 real serving configurations on one H100 PCIe (1.85 TB/s measured copy bandwidth): two stack modes, five architectures, real `bitsandbytes` NF4 weights, a real quantized int4 KV cache, and real speculative decoding (`docs/results/clock_channel*.json`). The premise survives every row; the naive *absolute* test survives none of them.

- **The floor is a stack property, not a spec sheet.** Observed ms/token is $4.15\text{ ms} + \text{bytes}/1.22\text{ TB/s}$: the slope is physics (66% of the card’s measured copy bandwidth), but it sits on a 4.15 ms additive constant that is pure serving stack — the same GPU and weights one stack down (plain eager) cost $\sim 30\text{ ms}$ per token, *flat* in bytes. Nor can the floor be computed from a config file: a slope law fitted on five configurations misses five unseen architectures by 26% mean and 48% worst. An auditor that does not know the stack cannot convert a roofline into an accusation.

- **Fewer bytes is not proportionally less time.** Real NF4 weights save $2.55\times$ the bytes and deliver 14% of the predicted time saving — dequantization spends the rest — and are *slower* than bf16 at every batch size ≥ 4 . At these scales weight quantization buys the provider capacity, not speed, and a latency channel cannot see a capital saving.
- **A byte-saving KV cache is not a latency deviation.** A real int4 KV cache, identical positions and $4\times$ fewer cache bytes, is *slower*: the context slope steepens $1.6\times$. Single-query attention is per-position, not per-byte.
- **Honest servers already live below the “floor”.** Under real speculative decoding — an honest provider by construction, since greedy acceptance emits the target’s own continuation — **73% of the visible inter-token gaps are 0 ms**. Tokens arrive in clumps, so any minimum-gap statistic accuses essentially every real provider.

What survives: subtract. All three obstacles — the unknown stack constant, co-tenancy, sub-floor gaps — attack absolute tests, and none of them attacks a *difference*. The surviving verifier sends two probes matched in everything but prompt length and reads $D = \text{ITL}(\text{ctx}_{hi}) - \text{ITL}(\text{ctx}_{lo})$: the stack constant and any constant network offset cancel exactly. Two measured properties make it deployable. First, co-tenancy is a *one-sided* nuisance — load only pushes D up, toward the honest side — so a busy datacenter costs the auditor power, never a false accusation. Second, the verifier is self-calibrating: an endpoint’s own honest slope re-measured in a fresh process drifts by 0.01%, three orders of magnitude tighter than the cross-architecture law, so the floor is calibrated on the auditor’s own honest traffic rather than derived from a spec sheet — which also means the test detects a provider that *starts* truncating, the realistic threat.

Priced through the same protocol. A provider billing for 32,768 tokens of context while attending a window W , scored through the identical `harness.evaluate` as every token detector (standardized pAUC @ FPR $\leq 0.5\%$, honest calibration split, ratio ceiling enforced), under a pessimistic $\sigma = 50$ ms of auditor-side wire jitter:

window held	d' per probe pair	pAUC @ 32 pairs	tokens for pAUC 0.90
512	0.89	0.996	36
2,048	0.82	0.987	42
8,192	0.68	0.940	62
16,384 (half)	0.43	0.714	152

Thirty-six to 152 tokens of stream per verdict, against the token channel’s 2,013 for 4-bit weights and 35,066 for an fp8 KV cache — and read at prefill instead of in the decode gap, the same premise is four orders of magnitude cheaper again on the substitution row (`docs/results/clock_algos.json`). Measured pAUC tracks the same $d'\sqrt{b}$ law as Section 3.3, so the channel is priced by the report’s existing cost model with nothing added. Evasion is possible but not free in the way the original cheat was: to hide, a truncating provider must pad each long request by exactly what truncation saved — up to 87% of the honest per-token time — keeping the memory and giving back all of the speed. A differential clock does not prove honesty; it converts the cheat from free margin into a real cost, which is what an enforcement instrument needs.

7 What a selective audit really costs

If a full audit is too expensive, audit some tokens. The natural implementation ranks tokens by a cheap proxy signal and recomputes the top fraction. The fraction of tokens audited then gets

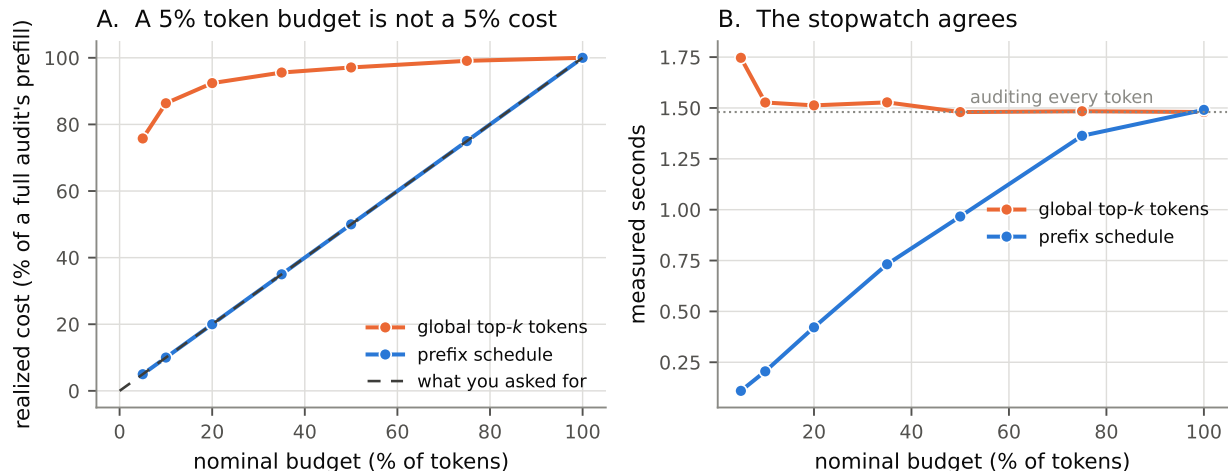


Figure 2: The audit-cost result, and the one clean positive in the study. **A:** a global top- k audit’s realized prefill cost against its nominal token budget; the prefix schedule lies on the diagonal (it is hidden under the dashed reference line, which is the point). **B:** wall clock. Top- k at a 5% budget is slower than auditing every token. Qwen3-1.7B, 64 sequences $\times$ 128 tokens.

reported as the cost. **It is not the cost**, and the gap is large enough to invert conclusions.

Reading M ’s logits at generated position j requires a prefill over `prompt + claimed[:j]`. So a sequence’s audit cost is set by its *deepest* audited position, and any sequence touched at all re-pays for its prompt. A global top- k rule scatters its audit across essentially every sequence, which means it pays nearly every prefill in order to score a few tokens. Measured on a backend instrumented to prefill lazily and refuse to serve an unpaid row (64 sequences $\times$ 128 tokens, Qwen3-1.7B):

nominal budget	global top- k		prefix schedule	
	realized cost	seconds	realized cost	seconds
5%	75.8%	1.75	5.0%	0.11
10%	86.4%	1.53	10.0%	0.21
20%	92.4%	1.51	20.0%	0.42
50%	97.1%	1.48	50.0%	0.97
100%	100%	1.48	100%	1.49

The sharpest number here needs no statistics: **a “5%” top- k audit takes 1.75 seconds, and auditing every single token takes 1.48**. The reported saving was overstated by up to $15\times$. The fix is a scheduler that admits contiguous per-sequence prefixes greedily by value per unit of *marginal* cost — depth is cheap, breadth is expensive — which spends exactly what it is budgeted and runs $16\times$ faster at a 5% budget. Plotted against realized cost rather than nominal budget, one prefix-scheduled operating point (75% of a full audit, AUC $0.788 \pm .043$) costs strictly less *and* detects 0.12–0.20 AUC more than four separate top- k points, every one of which the naive accounting would have called cheaper. Those four separate by 1.5–4.4 combined standard deviations and only two clear 2sd, so the individual comparisons are directional; the cost difference underneath them is not.

7.1 [new] Is the cost result about the attack, or about the geometry?

Everything above was measured against a single deviation (`quant_2bit`). Two of its claims could in principle be attack-dependent, and they are worth separating. The *accounting* claim — that a nominal token budget badly misstates realized prefill — should not depend on the attack at all, because prefill cost is a function of prompt lengths, sequence count and audit depth, and none of those know what is being detected. The *Pareto* claim — that a prefix-scheduled point can cost less and detect more — could easily depend on it, because it depends on where the evidence sits in a sequence. We re-ran the whole experiment on two further attacks, one forward-pass (`kv_fp8`) and one sampler-side (`bug_k32`), holding the model, the prompts, the value signal and the protocol fixed.

attack	global top- k @ 5%		prefix @ 5%		speed-up	full audit	
	cost	sec	cost	sec		sec	AUC
<code>quant_2bit</code>	75.8%	1.75	5.0%	0.11	16.0×	1.48	0.823 ± 0.050
<code>kv_fp8</code>	76.9%	1.51	5.0%	0.10	14.5×	1.56	0.507 ± 0.005
<code>bug_k32</code>	78.1%	1.31	5.0%	0.09	14.4×	1.14	0.601 ± 0.042

Table 4: [new] The accounting gap is a property of the schedule, not of the deviation. Qwen3-1.7B, 64 sequences $\times$ 128 tokens, 5 protocol seeds; “cost” is realized prefill as a fraction of a full audit’s. All three attacks are scored on identical prompts with an identical scheduler.

The accounting result replicates exactly, and that is the expected outcome. At a nominal 5% budget the global top- k audit realizes 75–77% of a full audit’s prefill on every attack, and the prefix schedule realizes 5.0% on every attack, for a wall-clock speed-up of 14–16 $\times$. The spread across attacks (75.8% versus 76.9%) is smaller than the spread across protocol seeds within any one of them. This is the sense in which the result “contains no statistics”: re-running it against a different deviation is a null experiment by construction, and it comes back null.

The detection result is only testable where there is detection to allocate, and this is where the second attack is informative in a way we did not plan. Under `kv_fp8` every point on both curves — 14 operating points across two schedulers — lies within 0.04 of chance, including the full audit (0.507 ± 0.005). There is no signal at any budget, so there is nothing for an allocation rule to allocate well or badly, and the Pareto comparison is undefined rather than negative. That is the same wall Sections 3.4 and 5 hit from two other directions: at 10^4 tokens an fp8 KV cache is simply below the recompute detection floor. We note it because a reader who saw only the `kv_fp8` panel might conclude the scheduler had stopped working, when what stopped was the detector.

attack	cost	prefix AUC	full audit	top- k dom.	AUC gap	sep. (sd)
<code>quant_2bit</code>	35%	0.811 ± 0.069	0.823 ± 0.050	5 of 7	0.07–0.23	0.8–3.2 (1)
<code>bug_k32</code>	35%	0.610 ± 0.047	0.601 ± 0.042	7 of 7	0.01–0.10	0.1–2.0 (2)

Table 5: [new] The honest Pareto plane, restricted to the attacks that are detectable at all at this scale (`kv_fp8` is excluded: its full audit reads 0.507 ± 0.005 , so every comparison over it is noise against noise). “Dominated” counts global top- k operating points that cost strictly more *and* detect strictly less than the single prefix-scheduled point in column 2.

Where there is signal, the ordering replicates — directionally. `bug_k32` is barely detectable at this scale (a *full* audit reads 0.601 ± 0.042), which makes it a demanding test rather than an easy one. Its best prefix point spends 35% of a full audit’s prefill and reads 0.610 ± 0.047 —

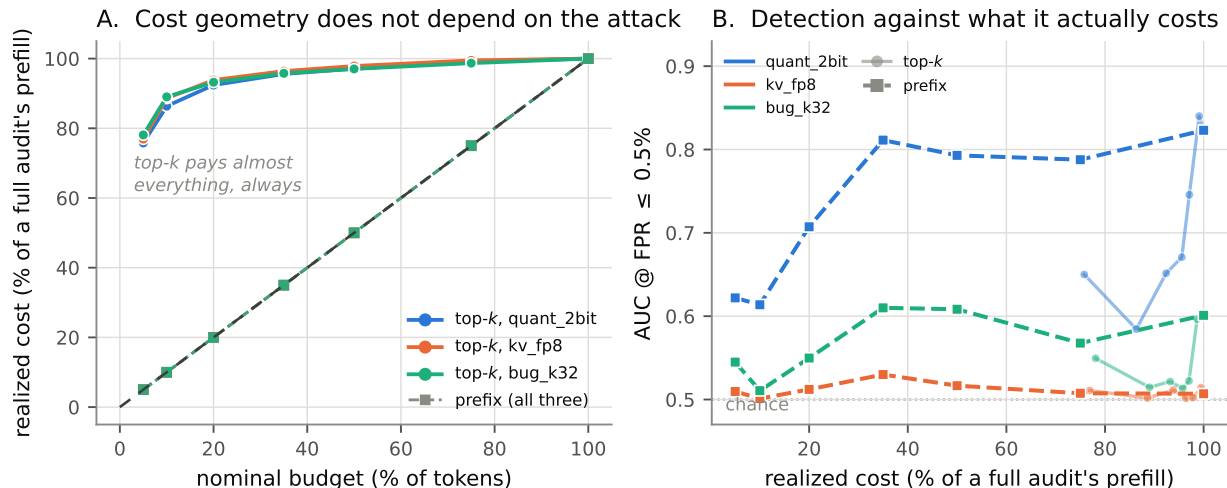


Figure 3: [new] **A:** the accounting gap for every attack, with the prefix schedules (dashed) on top of one another along the diagonal. The curves are indistinguishable because prefill cost does not depend on what is being detected. **B:** the same runs in the only plane that is honest about price — detection against realized cost, not against nominal budget. Under `kv_fp8` both schedulers sit flat on the chance line at every price.

matching or beating *every* global top- k point, all seven of which cost between 78% and 100%. The shape is the same as `quant_2bit`'s: the scheduler reaches the full-audit AUC at roughly a third of the price, and the top- k curve does not reach it at any price below 100%.

We are deliberate about how strong a claim that is. The cost half is not in doubt: a 2–3 $\times$ difference in realized prefill and a 14–16 $\times$ difference in wall clock are far outside any seed-to-seed variation. The detection half is directional. Taking the two arms as unpaired and combining their standard deviations, the individual dominances separate by 0.1–3.2 sd, and only three of the twelve clear 2sd (Table 5, last column). What replicates is the *pattern* — on both detectable attacks, a prefix point at about a third of the cost sits at or above the entire top- k curve — not any single pairwise comparison. Five to nine protocol seeds on an 8,192-token pool is not enough to resolve one, which is the limitation of Section 10 showing up exactly where we would expect it.

A cheap sanity check on the exclusion in Table 5: run the same dominance count on `kv_fp8` anyway and the prefix point “dominates” 7 of 7 top- k points too — with gaps of 0.002–0.028 and separations below 1.6sd. Counting dominated points is meaningless when both curves are flat on chance, which is precisely why the attack is excluded rather than reported as a third success.

8 Two things that did not transfer

Given a budget to spend, where should it go? Two principled answers were implemented and both lost to a one-line heuristic. We report them because the pattern they form is more useful than either result alone.

8.1 A learned, calibrated per-token value head

The first ports the confidence head from DSpark [2]: a logistic model over nine cheap proxy-side features (entropy, Rényi-2, tie margin, log top-1, top-5 mass, surprisal, log rank, log support, relative position), trained with cross-entropy against per-token *sensitivity* — how much the detector’s score moves under a generic zero-mean logit probe — using **honest data only**, then temperature scaled.

It is a strict generalization of the hand-written signals: two of its features reproduce the existing `entropy` and `tie_margin` rules exactly.

It does not win. Against `entropy`, the library’s existing default, it leads one budget row out of six by well under two standard deviations, and `entropy` leads two. Used instead as the value term inside the prefix scheduler — the role DSpark actually uses it for — it leads one row of seven, `entropy` three, with no pair separating by 2sd anywhere. Its calibration stage buys nothing: the fitted temperature is 0.975, and on a held-out honest split the expected calibration error gets *worse* ($0.0190 \rightarrow 0.0246$). A head trained with cross-entropy against the same label the reliability curve scores is already calibrated.

The informative part is what it learned. It puts its largest positive weights on tie-ness (+0.86) and proxy surprisal (+0.80), and a **negative** weight on entropy (−0.28) — while `entropy` is the best allocator in the table. The head is not failing to fit; it is fitting the wrong target. Per-token sensitivity is not the quantity that maximizes batch-level detection.

8.2 [new] The correct target, derived — and it still loses

That diagnosis has a clean fix on paper. For a weighted, centered batch statistic, the detection power of an audited set A is

$$\delta^2(A) = \frac{b}{n} \sum_{t \in A} I(t), \quad I(t) = \frac{\Delta(t)^2}{v(t)},$$

with $\Delta(t)$ the mean shift the deviation causes at position t and $v(t)$ that position’s *honest* variance — the local nondeterminism. Sensitivity is the numerator only. A near-tie position is sensitive *and* noisy, so ranking on Δ buys both, which is exactly the failure above. The theory also names the optimal aggregator: the matched filter $w = \Delta/v$, in place of the plain mean every number in this study is computed with.

We ran this at full scale for this report (Qwen3-1.7B, 64 prompts $\times$ 128 tokens, batch 400 at a 9.8% ratio, 9 protocol seeds), crossing 7 allocation rules with 3 aggregation rules, including oracle arms that read Δ and v off labelled honest/attack pairs so that a negative result would be diagnosable.

It loses, and so does the oracle. Holding the allocation at the derived I ranking, the matched filter is *below* the plain mean at every budget (at a 5% budget, $-0.078 \pm .008$, $t = -10.4$ paired by seed), and substituting oracle weights does not rescue it ($-0.142 \pm .024$, $t = -6.0$ against the plain mean). Two of the comparisons deserve to be stated plainly: even with labels for free, the derived aggregator is worse than averaging, and the theory arm that predicts the whole detection-versus-budget curve from the information captured overshoots the oracle measurement by 0.24 AUC on average.

Why is the useful part, and Figure 4B is the answer. Fitting the same nine features on labelled pairs — labels for free, no estimation error — predicts the log honest variance well ($R^2 = 0.73$) and the signal not at all ($R^2 = 0.002$). Since $I = \Delta^2/v$ divides a quantity the features cannot estimate by one they can, the resulting information estimate is not merely noisy but **anti-correlated** with the truth (Spearman -0.92). The derivation is right; the inputs it needs are not recoverable from what a verifier can cheaply see. Where the deviation will show up is, on this evidence, roughly unpredictable from the proxy’s view of a position — and a matched filter built on an anti-correlated weight estimate is worse than no weighting at all.

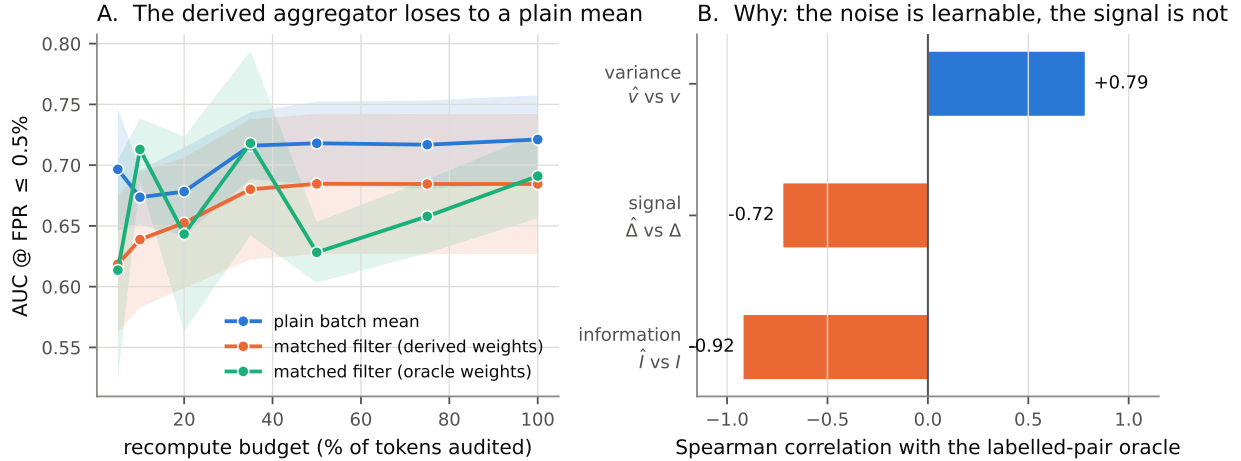


Figure 4: [new] Information-directed verification at full scale (Qwen3-1.7B, `quant_2bit`, 8,192-token pool, batch 400, 9 seeds). **A:** allocation held fixed at the derived I ranking so only the aggregation rule varies; the plain mean is above the derived matched filter at every budget (bands are ± 1 sd; the paired-by-seed test gives $t = -10.4$). **B:** the diagnosis. Rank correlation of each estimated quantity against its labelled-pair oracle: the cheap features track the local noise well and get the signal *backwards*, so the information estimate they imply is strongly anti-correlated with the truth.

8.3 The pattern

The prefix scheduler is a statement about prefill geometry: it holds for every value signal, contains no statistics, and delivers $16\times$. The confidence head and the matched filter are statements about where the evidence is, and both fail. In selective verification, **the cost side transfers and the signal side does not** — so the engineering worth doing is on the accounting, not on the estimator. Anyone porting speculative-decoding machinery into a verification setting should take the scheduler and skip the head.

9 What we would actually deploy

1. **Cascade, and put the cheap tier first for the right reason.** A verifier-side proxy panel catches capability downgrades — the deviations with the most compute saved in them — at a small fraction of one forward pass. Use it as a continuous first-line filter, not as a verifier.
2. **Keep recomputation for what only recomputation can see.** Quantization, fp8 caches, and sampler deviations are recompute-dominant; no cheap statistic reached them in any experiment here. Spot-check with it rather than running it continuously.
3. **Read the clock before spending FLOPs — differentially, never absolutely.** The arrival times of a stream already bought catch every deviation that attends fewer positions than billed (context truncation, window eviction, stale prefix reuse) at 36–152 tokens per verdict, with zero audit compute and no cooperation. Calibrate the slope on your own honest traffic to that endpoint (it is stable to 0.01%); never test against a roofline or a minimum gap — the stack constant and honest token clumping make every absolute latency test either uncomputable or an accusation machine (Section 6).
4. **Use a portfolio of detectors, not the best one.** Each recompute score is structurally blind to a different class of deviation, and which one wins is set by the attack, not by the model — the winning detector was identical in all six rows of Table 1 across an $8.8\times$ change

in pool size. Run them together. `token_toploc` deserves particular attention because it needs no seed synchronization. (The cross-family sweep’s claim that a portfolio lifts the floor to 0.78 is measured at an invalid ratio; treat it as an ordering, not a level.)

5. **Budget in prefill tokens, and schedule prefixes.** A token-fraction budget is not a cost. Depth is cheap, breadth is expensive.
6. **Spend the tuning effort on pool size, not on the estimator.** Effect sizes here are small ($d' \approx 0.07$ for 4-bit quantization); the binding constraint on a trustworthy verdict is how many honest tokens you have, and every attempt to be cleverer than a plain mean failed. Size the pool before running anything: d' is cheap to measure on a few thousand tokens and tells you the batch — and hence the pool — the verdict will require (Section 3.3).
7. **Report the batch/pool ratio next to every AUC.** It is one number, and without it a detection AUC is not interpretable.

10 Limitations

- **Model scale.** Everything runs on a single H100, so the reference models are 0.13B–8B. The economically interesting case is a frontier model, where the attacker’s incentive is larger, honest nondeterminism may differ, and the proxy-to-target ratio is much smaller.
- **Attacks are mostly simulated at the logit level.** Temperature, seed and sampler-bug attacks are real specification changes, but quantization and fp8 caches are applied as perturbations on top of real logits rather than by loading genuinely quantized weights. A faithful deterministic quantization is sparse and heavy-tailed, structurally unlike an i.i.d. Gaussian perturbation, even when matched on mean divergence — so the quantization numbers here should be read as a model of the attack, not a measurement of it. The one genuine nf4 result we have (in the proxy panel study) also reads chance, which is at least consistent.
- **The verifier is passive.** No experiment here has the attacker adapting to the specific verifier it faces, beyond the sampling-knob searches in the proxy study. An attacker who can query the deployed detector is a strictly stronger adversary, and the sampling-only evasions already found suggest that is where the remaining risk is.
- **Small-sample statistics remain the weak axis.** Five to nine protocol seeds over pools of 10^4 tokens can separate triage from random allocation but cannot rank two triage signals against each other. Several comparisons in this report are directional rather than resolved, and we have tried to label which.
- **Most of the record is still measured over the ceiling.** We re-measured the headline grid, the pool-scaling prediction and five cross-family cells, which is where the load-bearing claims sit. The remaining 58 cells of the cross-family sweep, the proxy-panel study and the confidence-head tables were not re-run at a valid ratio, and the correction we did apply was large — a median 0.14 AUC and up to 0.46 — so their absolute levels should be read with the same suspicion. Section 3 is a statement about this report as much as about the testbed.
- **One prompt domain, one GPU.** Cross-GPU and cross-datacenter benign variation — arguably the most important false-positive source in a real deployment — is untested, because there was only one accelerator available.
- **The clock channel is measured on research stacks, and the wire is swept rather than measured.** Section 6’s grid runs HF eager and HF + CUDA graphs on one H100 PCIe; a production stack (vLLM, TensorRT-LLM) will lower the additive constant and may move

the context term back toward bandwidth-bound, which could restore fp8-KV detectability. The auditor-side jitter σ is swept (the quoted prices use a pessimistic 50 ms), not measured against public endpoints, and no provider here shapes or pads traffic against the slope test — the padding counter-move is priced but not played.

References

- [1] A. Karvonen et al. *DiFR: Inference Verification Despite Nondeterminism*, 2025. <https://arxiv.org/pdf/2511.20621>
- [2] *DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation*, 2026. <https://arxiv.org/abs/2607.05147>
- [3] Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding*, 2022. <https://arxiv.org/abs/2211.17192>
- [4] J. Clymer et al. *Lessons from building a model-organism testbed*, 2025.